FADENA
FACULTAD DE DEFENSA NACIONAL

Universidad de la Defensa Nacional

Facultad de Defensa Nacional

Licenciatura en Ciberdefensa

Sistemas de Tratamiento de Datos

Modelos de datos y abstracción

Año: 2025

Modelos de Datos

El modelado de datos es el primer paso en el diseño de una base de datos, implica el proceso de crear un modelo de datos específico para un problema o necesidad determinada del mundo real. Este proceso es iterativo y progresivo, iniciando en un entendimiento básico del dominio del problema, y llevando al modelo a un nivel más detallado a medida que se adquiere conocimiento. En este sentido, se entiende como un esfuerzo colaborativo que involucra a diversas partes interesadas, desde los especialistas del negocio para el que se está definiendo el modelo, usuarios finales, y hasta arquitectos de sistemas.

Un modelo de datos se concibe como la representación visual y abstracta de elementos de la realidad y las conexiones inherentes entre ellos (se puede entender como la abstracción de una parte del mundo real). Su propósito primordial es definir los datos que una aplicación o sistema de información requiere para lograr sus objetivos de negocio. Un modelo de datos bien diseñado establece una serie de beneficios clave, que incluye la reutilización eficiente de los datos, la facilidad de mantenimiento, y una mejora en la adaptabilidad y escalabilidad del sistema.

Los modelos de datos actúan como un puente crítico entre las necesidades abstractas del negocio y las implementaciones técnicas concretas. Al estructurar visualmente los datos de una manera comprensible tanto para los actores empresariales como para los técnicos, estos modelos aseguran una alineación precisa entre los

objetivos estratégicos de la organización y los sistemas de información subyacentes. Esta función de traducción minimiza las interpretaciones erróneas, reduce la fricción durante el desarrollo y, en última instancia, conduce a soluciones tecnológicas más efectivas.

Importancia de los modelos de datos:

- **Organización y coherencia:** Permiten estructurar los datos de manera lógica, asegurando su consistencia y facilitando su comprensión.
- **Comunicación y Colaboración:** Sirven como un lenguaje común entre desarrolladores, diseñadores de bases de datos, analistas de negocio y usuarios finales. Un modelo bien definido asegura que todos tengan la misma comprensión de los datos y sus relaciones.
- **Eficiencia en el Almacenamiento y Recuperación:** Un modelo optimizado permite almacenar datos de manera eficiente, minimizando la redundancia y facilitando consultas rápidas y efectivas.
- **Integridad de los Datos:** Definen reglas y restricciones que garantizan la validez y precisión de los datos. Por ejemplo, aseguran que un campo numérico contenga sólo números, o que una fecha sea válida.
- **Flexibilidad y Adaptabilidad:** Si bien se busca estabilidad, un buen modelo debe permitir cierto grado de flexibilidad para adaptarse a

nuevos requisitos o cambios en la información a lo largo del tiempo.

- **Base para el Desarrollo de Aplicaciones:** Son el cimiento sobre el cual se construyen las aplicaciones. La forma en que se modelan los datos impacta directamente en la complejidad y el rendimiento de las funcionalidades de una aplicación.

- **Análisis y Toma de Decisiones:** Un modelo de datos bien estructurado facilita la realización de análisis complejos, la generación de informes significativos y, en última instancia, la toma de decisiones informadas.

Tipos de Modelos de datos (evolución histórica):

La búsqueda de una mejor gestión de los datos ha llevado al desarrollo de varios modelos que intentan resolver las limitaciones de los modelos anteriores. La evolución de los modelos de datos refleja una adaptación constante a las crecientes demandas de volumen, velocidad y variedad de datos. Los modelos pioneros sentaron las bases para las arquitecturas de datos modernas.

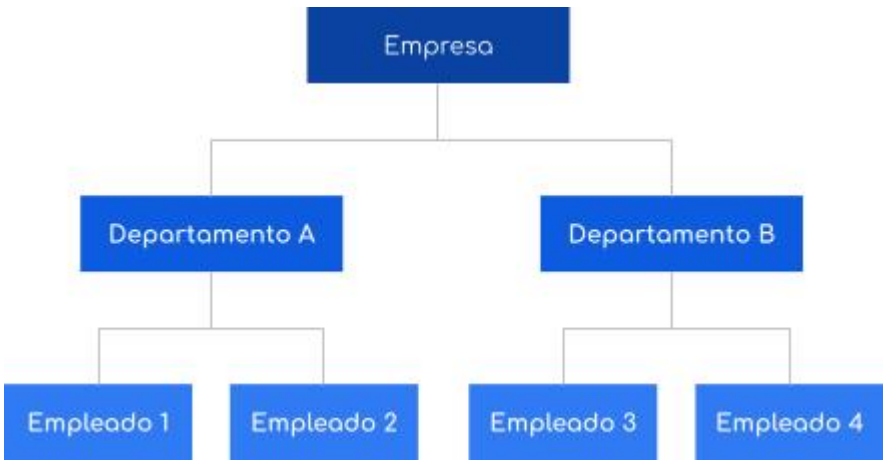
Modelo Jerárquico

Es el modelo predominante en los primeros sistemas de gestión de bases de datos. Fue desarrollado en la década de 1960 para gestionar grandes cantidades de datos en proyectos de producción, por ejemplo, en la fabricación de cohetes para el programa espacial Apolo de la NASA.

El ejemplo más destacado es el Information Management System (IMS) de IBM, que fue ampliamente utilizado en las décadas de 1960 y 1970.

Este modelo organiza los datos en una estructura de árbol, donde cada registro tiene un único padre y puede tener cero o más hijos. Es particularmente eficaz para representar relaciones uno a muchos (1:N), donde cada padre puede tener múltiples hijos, pero cada hijo tiene un sólo padre. Los registros se interconectan mediante relaciones entre "padres" e "hijos", que a menudo se implementan con punteros físicos, permitiendo un acceso rápido a los datos dentro de la jerarquía.

- **Nodo raíz:** Es el punto de entrada superior de la jerarquía. No tiene padre y todos los demás nodos dependen directa o indirectamente de él.
- **Nodos internos:** Son todos los nodos que tienen tanto un padre como al menos un hijo. Representan niveles intermedios de la jerarquía.
- **Nodos hoja:** Son los nodos más bajos de la jerarquía. Tienen un padre, pero no tienen hijos, y a menudo contienen los datos detallados.



- **Ventajas:**
    - Simplicidad conceptual: Ofrece una estructura simple y fácil de entender para problemas que se ajustan naturalmente a esta organización (similar a un árbol familiar o un organigrama empresarial).
    - Acceso eficiente en estructuras estáticas: Para sistemas con relaciones de datos estables y bien definidas, permite un acceso rápido a los datos debido a su estructura piramidal y el uso de punteros físicos, ya que las rutas están predeterminadas.

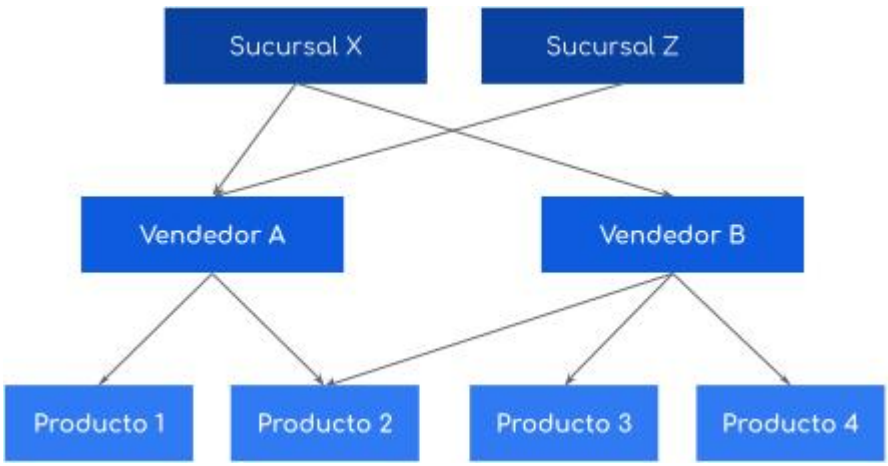
- **Desventajas:**
    - Baja flexibilidad: Su rigidez es una limitación significativa; es complicado de modificar una vez establecido. Cambiar la estructura jerárquica o añadir nuevas relaciones a menudo requiere reestructurar la base de datos completa.
    - Dificultad para representar relaciones complejas: Las relaciones "muchos a muchos" no se pueden modelar directamente, lo que limita su aplicabilidad en escenarios de datos interconectados y dinámicos. Para simular estas relaciones complejas a menudo se duplican los datos, lo que lleva a ineficiencias de almacenamiento y posibles inconsistencias (redundancia de datos).

Por estos inconvenientes es raramente visto en sistemas modernos. Sin embargo su influencia perdura en ciertas estructuras de datos, como XML y JSON (que son inherentemente jerárquicos).

Modelo de Red

El modelo de red es una evolución del modelo jerárquico, diseñado para superar su limitación principal al permitir relaciones muchos a muchos (M:N), en una estructura tipo árbol que permite múltiples padres para un registro. Utiliza nodos para representar los registros de datos y enlaces (punteros) para definir las relaciones entre ellos. Se basa en la teoría matemática de conjuntos, donde cada "conjunto" consiste en un registro "propietario" (padre) y uno o más registros "miembro" (hijos). La clave es que un registro puede ser miembro en múltiples conjuntos, lo que le permite representar relaciones complejas de manera más flexible.

Este modelo fue muy popular en la década de 1970, especialmente después de su formalización por el grupo CODASYL (Conference on Data Systems Languages).



Ofrece una estructura mucho más flexible que el modelo jerárquico al permitir múltiples relaciones entre nodos, lo cual es particularmente útil en escenarios donde los datos no pueden organizarse fácilmente en una

estructura de árbol. Esta flexibilidad permite representar relaciones más complejas que el modelo jerárquico, reduciendo la redundancia.

Más allá de estas ventajas respecto al modelo jerárquico, su complejidad inherente al manejar múltiples relaciones requiere mayor trabajo en la administración y gestión de los datos, con procesos de diseño y consulta (navegación) muy complicados.

Modelo Relacional

Es el modelo de datos más extendido y utilizado en la actualidad. Organiza los datos en tablas bidimensionales, también conocidas como "relaciones", que se componen de columnas (atributos) y filas (tuplas). Las relaciones entre diferentes tablas se establecen y gestionan mediante el uso de claves primarias (identificadores únicos dentro de una tabla) y claves foráneas (referencias a claves primarias en otras tablas).

Tiene una gran simplicidad conceptual a partir de este uso de tablas (son muy intuitivas); y a partir de la normalización de estas se puede reducir la redundancia de datos, lo que mejora la flexibilidad, adaptabilidad y escalabilidad del sistema, asegurando que cada dato sea atómico. Además, presenta una alta integridad de datos gracias a las claves y restricciones.

Presenta un potente lenguaje de consulta (SQL) como estándar para interactuar con las bases de datos.

Es la base de los sistemas de gestión de bases de datos (SGBD) tradicionales y ampliamente populares como MySQL, PostgreSQL, Oracle Database y Microsoft SQL Server. Se utiliza en la mayoría de las aplicaciones del mundo real, incluyendo la gestión de inventario en la industria minorista, registros médicos electrónicos, sistemas bancarios y comerciales (gestión de cuentas, transacciones), plataformas de comercio electrónico (gestión de clientes, pedidos, catálogos), etc.

Sin embargo, puede ser menos eficiente para el manejo de datos no estructurados o semiestructurados. La normalización, aunque beneficiosa para la integridad, puede llevar a la segmentación de datos, resultando en consultas más complejas que involucran múltiples tablas (joins) y, potencialmente, tiempos de latencia más largos. Su escalabilidad horizontal puede ser más restrictiva en comparación con las bases de datos NoSQL, y modificar el esquema en sistemas relacionales puede ser un proceso laborioso y costoso, a menudo requiriendo tiempo de inactividad.

Modelo Orientado a Objetos (OO)

Este modelo representa los datos como "objetos", que son unidades de software reutilizables que encapsulan tanto los datos (atributos) como las operaciones (métodos) que pueden aplicarse a esos datos. Facilita la integración con lenguajes de programación orientada a objetos (OOP). Se le considera un modelo "post-relacional" o "híbrido" porque, si bien puede incorporar estructuras tabulares, no se limita a ellas y permite una representación más rica y compleja de la realidad.

Dentro de los modelos OO, existen especializaciones como las bases de datos multimedia y las bases de datos de hipertexto. ObjectDB y db4o son ejemplos de bases de datos orientadas a objetos.

Ofrece una gran capacidad de modelado, con conceptos como el encapsulamiento y la herencia. Proporciona flexibilidad al permitir estructuras cambiantes mediante la adición de subclases y manipula objetos complejos de forma rápida y ágil, ya que la estructura de la base de datos se basa en referencias lógicas entre objetos. Permite una alta velocidad de procesamiento al no requerir el reensamblaje de objetos en cada acceso, promueve la extensibilidad (construcción de nuevos tipos de datos) y mejora los costos de desarrollo a través de la reutilización de código.

Una de las principales debilidades es la carencia de un modelo de datos universalmente aceptado y la falta de estándares en la industria, su adopción generalizada ha sido limitada. La estructura de una BDOO es más compleja y difícil de entender que la de una base de datos relacional.

Modelos NoSQL

Los modelos NoSQL (Not Only SQL) representan una categoría diversa de bases de datos diseñadas para abordar las limitaciones de los modelos relacionales en escenarios específicos, particularmente con grandes volúmenes de datos no estructurados o semiestructurados. Estos modelos son inherentemente escalables horizontalmente, flexibles y ofrecen un rendimiento rápido.

Algunos de los tipos comunes de modelos NoSQL incluyen:

Clave-Valor: Almacena datos como pares simples clave-valor, (Redis, Amazon DynamoDB).

Documentos: Organiza datos en documentos (comúnmente JSON o BSON), ofreciendo un esquema flexible que permite almacenar cualquier tipo de datos. (MongoDB, Couchbase).

Orientadas a Grafos: Utilizan estructuras de grafos (nodos, bordes, propiedades) para representar datos y sus relaciones, para análisis de redes complejas. (Neo4j, Amazon Neptune).

Orientadas a columnas: Almacenan datos en columnas en lugar de filas, lo que es eficiente para consultas analíticas que solo necesitan ciertas columnas (agregaciones). (Apache Cassandra, Google BigQuery)

Las ventajas de los modelos NoSQL incluyen su capacidad para manejar grandes volúmenes de datos no estructurados o semiestructurados, alta escalabilidad horizontal (añadiendo más nodos al clúster), rendimiento superior para operaciones de lectura/escritura distribuidas, y un diseño sin esquema que permite flexibilidad y desarrollo ágil. Son ideales para aplicaciones que requieren procesamiento de datos en tiempo real y entornos donde los requisitos de datos evolucionan rápidamente. Ejemplos de uso incluyen Netflix, Meta, Amazon y LinkedIn para gestionar transacciones, comentarios, recomendaciones y perfiles de usuario.

Sin embargo, presentan desventajas como la falta de estandarización entre las diferentes bases de datos NoSQL, lo que puede complicar la elección y el aprendizaje. Su madurez relativa en comparación con SQL

puede reflejarse en la disponibilidad de herramientas y soporte. Pueden presentar inconsistencia de los datos debido a modelos de consistencia eventual (priorizando disponibilidad y tolerancia a particiones sobre consistencia fuerte). Además, pueden requerir un mantenimiento más complejo debido a su arquitectura distribuida y tienen un soporte limitado para consultas y transacciones complejas en comparación con las bases de datos SQL.

El Concepto de Abstracción en los Modelos de Datos

Un objetivo fundamental de los sistemas de bases de datos es proporcionar a los usuarios una vista abstracta de los datos, lo que implica ocultar los detalles complejos de cómo se almacenan y manipulan los datos, información que los usuarios no necesitan conocer. Los modelos de datos actúan como la herramienta principal para lograr esta abstracción, presentando diferentes niveles de complejidad en la representación de la información.

La arquitectura ANSI/SPARC es un marco conceptual que permite visualizar una base de datos en tres niveles distintos de abstracción: el nivel físico, el nivel lógico y el nivel de visión (o externo).

La abstracción sustenta la robustez, mantenibilidad y longevidad de los sistemas de bases de datos. Al desacoplar diferentes aspectos de la representación y el almacenamiento de datos, permite realizar modificaciones en un nivel sin requerir cambios generalizados en niveles superiores. Esta separación de preocupaciones reduce significativamente los costos de desarrollo y mantenimiento, mejora la

flexibilidad del sistema y permite que diferentes partes interesadas (usuarios de negocio, arquitectos de datos, administradores de bases de datos) se centren en sus responsabilidades específicas.

Modelo de Visión (o Externo)

Este es el nivel más alto de abstracción. Es la vista que tienen los usuarios finales del entorno de datos, ofreciendo una representación de alto nivel de las entidades, sus atributos y las relaciones fundamentales, enfocándose en una unidad de negocio específica.

Las organizaciones por lo general están divididas en diferentes unidades de negocio (por ejemplo ventas, finanzas, marketing, etc.). Cada una está sujeta a restricciones y requerimientos específicos, y cada una utiliza un subconjunto de los datos generales de la organización. Este nivel de abstracción simplifica la interacción con cada uno de los usuarios, permitiendo definir diferentes vistas para la misma base de datos según las necesidades de cada grupo de usuarios o funcionalidad en particular.

En este nivel participan principalmente los usuarios finales y los programadores (desarrolladores) de las aplicaciones que hacen uso de la base de datos, a través del lenguaje de manipulación de datos (DML).

Modelo Lógico (o Conceptual)

Es un nivel intermedio de abstracción. Representa una visión global de toda la base de datos para toda la organización. Es decir, este modelo integra todas las vistas externas, definiendo las entidades, atributos y relaciones de los datos, así como restricciones de integridad y seguridad.

Es una descripción completa de la información que la base de datos debe almacenar, independientemente de cómo se organice físicamente o cómo la vean los usuarios individuales.

Describe la estructura lógica de la base de datos, en término de un modelo de datos. Estos incluyen tanto el modelo entidad-relación como herramienta gráfica; como el modelo relacional organizando los elementos de datos en tablas, columnas, claves, etc. En esta estructura se definen las relaciones y sus cardinalidades, nombres y tipos de datos específicos (int, varchar, etc.), nulidad, etc.

En este nivel participan principalmente los diseñadores de bases de datos y los administradores de bases de datos, a través del lenguaje de definición de datos (DDL).

Modelo Físico (o Interno)

Es el nivel más bajo de abstracción, se ocupa de los detalles de implementación de bajo nivel (más cercano al hardware) que son transparentes para los usuarios de los niveles conceptual y externo.

Representa la organización física real de los datos en los medios de almacenamiento, definiendo cómo se almacenan los datos (tipos de archivos, métodos de acceso, índices, particiones, ubicación en disco, etc.) para optimizar el rendimiento.

Si bien en este nivel los administradores de bases de datos también usan el DDL para definir el esquema interno; en muchos sistemas de gestión

de bases de datos modernas gran parte de esta configuración está automatizada o se gestiona a través de herramientas gráficas.

Mapeos entre Niveles

La relación entre estos tres niveles se gestiona mediante mapeos o transformaciones:

Mapeo Externo/Conceptual: Transforma las solicitudes y los resultados entre un esquema externo y el esquema conceptual. Permite que un usuario acceda a datos que son parte del esquema conceptual, pero presentados en un formato y con un subconjunto relevante para su vista.

Mapeo Conceptual/Interno: Transforma las solicitudes y los resultados entre el esquema conceptual y el esquema interno. Es responsable de traducir las operaciones lógicas sobre el esquema conceptual en operaciones físicas sobre el esquema interno y viceversa.

Independencia de Datos (Física y Lógica)

La independencia de datos es un concepto crucial que permite modificar el esquema en un nivel sin afectar los esquemas en niveles superiores.

Independencia física de datos:

Es la capacidad de modificar el esquema interno (físico) sin tener que cambiar el esquema conceptual, ni, por lo tanto, los esquemas externos o las aplicaciones que los utilizan. Permite realizar optimizaciones en la forma en que los datos se almacenan físicamente (por ej. cambiar el tipo

de archivo, añadir o eliminar índices, reorganizar el almacenamiento en disco, etc.) sin afectar la lógica de las operaciones.

Independencia lógica de los datos:

Es la capacidad de modificar el esquema conceptual (que información se almacena y cómo se relaciona) sin tener que cambiar esquemas externos o las aplicaciones que los utilizan. Permite a los diseñadores de bases de datos realizar cambios en la estructura lógica de la base de datos (por ej. añadir una tabla o campo, modificar un tipo de dato, etc.) sin afectar a todas las aplicaciones existentes que no necesitan ver o interactuar con esos cambios específicos. Se considera más difícil de lograr que la independencia física, ya que los programas de aplicación a menudo dependen en gran medida de la estructura lógica de los datos a los que acceden.

Esquema básico de la arquitectura ANSI/SPARC

